Trabajo Práctico Integrador «Food Store» — Segunda entrega

Base de Datos II · Tecnicatura Universitaria en Programación · UTN · Docente: Carlos Yácomo
Grupo H (Saferazi): Lautaro Fernandez, Daniela Díaz, Jesús Ramírez, Danilo Serrano, Elio Marí
Repositorio: https://github.com/lauticodemaster/UTN-TUPaD-Base-de-datos-2 · Unidades 4 a 8 · Noviembre de 2026
Motores: PostgreSQL 16+ con PL/pgSQL (pruebas de esta entrega en 18.6), MongoDB 8.0 y SQLite 3 (tablas STRICT)

Este informe resume lo hecho en las unidades 4 a 8 y cómo se verificó. Cada afirmación remite a un artefacto del repositorio (scripts, specs y salidas de ejecución). Las rutas se indican entre paréntesis.

#	Objetivo	Evidencia en el repositorio
1	Refactorización a FNBC validada contra las FD reales	`TP06 (Unidad 4)/sql/tp_fnbc_control_lote.sql`, `Parte-1/`, `ejercicio4.2_parte2/`
2	Migración sin pérdida, transaccional, especificada en Kiro, con reversión	`TP06 (Unidad 4)/specs/kiro/`, `sql/rollback_fnbc_control_lote.sql`, `evidencias/salida_fnbc_migracion.txt`
3	Desnormalización controlada documentada	`TP06 (Unidad 4)/sql/tp_desnormalizacion_top_categorias.sql`, `informe_u4.md`, `evidencias/explain_*.txt`
4	Usuarios, roles y permisos con mínimo privilegio	`TP07 (Unidad 5)/sql/roles.sql`, `docs/kiro_roles_spec/`, `docs/salida_verificacion_permisos.txt`
5	Auditoría activada; simulacro de acceso indebido reconstruido desde el log; contención	`TP07 (Unidad 5)/docs/informe_auditoria.md`, `informe_incidente.md`, `log_simulacro_*_anonimizado.txt`
6	Auditoría de permisos asistida por IA con decisión humana	`TP07 (Unidad 5)/docs/auditoria_ia_roles.md`, `BITACORA.md`
7	Plan de mantenimiento automatizado: respaldo programado, restauración probada, tareas rutinarias	`TP08 (Unidad 6)/Parte1…Parte4`, `Parte6_Programacion/programar_mantenimiento.ps1`
8	Simulacro de desastre contra reloj con informe post-incidente	`TP08 (Unidad 6)/Parte4_Simulacro/logs/`, `Parte5_Informe/informe_u6.pdf`
9–10	Modelo documental con consultas verificadas; comparación con la solución relacional	`TP09 (Unidad 7)/`, `informe/informe_u7.pdf`
11–12	SQLite especificado en Kiro; sincronización simulada con conflictos	`TP10 (Unidad 8)/docs/kiro/`, `movil/schema_sqlite.sql`, `movil/simulador.py`
13	Code review final asistido por IA con decisión por hallazgo	`TP10 (Unidad 8)/docs/informe.md` §5 (CR-01…CR-11), `code_review/`
14	Capa de datos completa, lista para la defensa	Este informe y el repositorio

1. Qué se implementó en cada unidad

Unidad 4 · Normalización avanzada y redundancia

- **Refactorización a FNBC** del esquema heredado `control_lote_almacen(lote, depósito, responsable)`. Dependencias funcionales: F1 `{Lote, Depósito} → Responsable` y F2 `Responsable → Depósito`. Claves candidatas: `{Lote, Depósito}` y `{Lote, Responsable}`. Como todos los atributos son primos, el esquema cumple 3FN pero no FNBC: el determinante de F2 no es superclave. Se descompuso en `responsable_deposito(responsable PK, depósito)` y `control_lote(lote, responsable)`, con la vista de compatibilidad `v_control_lote_almacen` (reunión natural). La descomposición es sin pérdida porque el atributo común es clave de `responsable_deposito`.
- **Migración** en una única transacción. Valida F2 *sobre los datos*, crea la estructura, migra y comprueba la equivalencia con `EXCEPT` en ambos sentidos antes del `COMMIT`. Si algo falla, aborta sola. El esquema heredado se conserva durante la convivencia y hay un script de reversión.
- **Salvaguarda de la dependencia no preservada:** la FNBC pierde F1, así que un trigger impide asignarle a un lote dos responsables del mismo depósito.
- **Desnormalización controlada** para el reporte «top 5 de categorías del día»: la tabla resumen `venta_categoria_dia(fecha, categoría) → total`, mantenida por triggers de delta sobre `detalle_pedido`, `pedido` y

`producto`, con TRUNCATE prohibido y un script de auditoría.

Unidad 5 · Administración y seguridad

- **Roles de grupo:** `rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes` y `rol_auditor`. **Roles de login:** `app_web`, `admin_datos` (dueño del esquema, sin SUPERUSER), `soporte_ops` y `reportes_bi`, con `CONNECTION LIMIT`.
- **Permisos por columna:** nadie, salvo el dueño, puede leer `usuario.contrasena`, y la aplicación puede registrar usuarios pero no elegir su `rol`, así que no puede crear un ADMIN. Nadie tiene DELETE porque las bajas son lógicas. `ALTER DEFAULT PRIVILEGES` deniega por defecto sobre los objetos futuros.
- **Autenticación:** `fn_autenticar` y `fn_resetear_password` con bcrypt (pgcrypto), `SECURITY DEFINER` y `search_path` fijo, bloqueo tras 5 fallos en 15 minutos y registro de cada intento en `auditoria_login`, sin la contraseña.
- **Auditoría nativa:** `log_connections`, `log_disconnections`, `log_statement='mod'` y un `log_line_prefix` con usuario, base, host y aplicación. La credencial humana de soporte se registra con `log_statement='all'`.
- **Simulacro de acceso indebido** y **anonimizador de logs**, con el mismo criterio determinista que `usuario_anon`.

Unidad 6 · Mantenimiento

- Respaldo lógico `pg_dump -Fc` con log de ejecución (scripts `.ps1` y `.sh`).
- `health_check.sql` para monitoreo y `maintenance.sql` para tareas rutinarias: estadísticas, limpieza y verificación de integridad.
- Restauración probada en una base descartable, con cronómetro y validación de conteos.
- **Programación** con el Programador de tareas de Windows, con equivalente cron:
  - respaldo diario a las 02:00 con retención de 14 días;
  - mantenimiento semanal;
  - prueba de restauración mensual.

  El equipo registró las tareas en su servidor y las ejecutó; los resultados fueron los esperados.

Unidad 7 · NoSQL

- Colección `foodstore.pedidos` en MongoDB, con un documento por pedido. Cliente y líneas van embebidos, y producto y categoría se guardan como snapshot histórico.
- Importes en `Decimal128`. Validador `$jsonSchema` + `$expr` que exige coherencia de subtotales y total.
- Consultas a)–d) verificadas contra las equivalentes en PostgreSQL, un experimento de divergencia de copias y un laboratorio de atomicidad y transacciones.

Unidad 8 · Bases de datos móviles e integración

- Base SQLite de la tablet, con tablas STRICT: `producto_cache`, borradores de pedido y `cola_sync`.
- Snapshot CSV del servidor y semáforo de sincronización: rojo = RECHAZADO, amarillo = CONFLICTO, verde = SIMULADO_APROBADO.
- Idempotencia por UUID de solicitud y lápidas para las bajas lógicas.
- Especificación en Kiro (requirements, design y tasks) y code review asistido por IA de los scripts de la cursada.

2. Cómo se probó

- **Respaldo y copia de trabajo.** Antes de cada cambio se hizo un `pg_dump -Fc` de `foodstore` (6 MB, 1,1 s). Todo se ejecutó sobre la copia `foodstore_test`, poblada con la carga masiva de TP03: 50.012 productos, 20.005 usuarios, 200.003 pedidos y 400.008 detalles. Es el procedimiento de `TP01/protocolo_seguridad.md`.
- **Transacciones reversibles.**
  - La migración FNBC se probó por el camino feliz, por el camino de aborto (un dato que viola F2) y con reversión posterior al `COMMIT` seguida de una nueva migración.
  - Las pruebas de anomalías, los 7 casos de sincronización de la tabla resumen, la verificación de permisos y el CHECK de CR-01 corren en transacciones que terminan en `ROLLBACK`.
- **Control negativo.** Una auditoría vacía solo prueba algo si se sabe que detectaría el error. Por eso se corrompió a propósito una celda de la tabla resumen y se comprobó que la auditoría la reporta.
- **Simulacro de acceso indebido.** El script `simulacro_incidente.sql` se ejecutó dos veces, antes y después de corregir el defecto que encontró la primera corrida. El log de cada corrida se anonimizó antes de analizarlo.
- **Simulacro de desastre.** Se borró la base y se restauró el último respaldo contra reloj. La primera vez fue sobre la base mínima de TP1 (TP8 de la cursada) y en esta entrega se repitió sobre la base poblada.

- **Conflictos de sincronización.** `simulador.py` ejecuta los casos C1–C7 sobre la base SQLite. Se volvió a correr para esta entrega y todos los casos dieron el resultado esperado.

3. Resultados obtenidos

3.1 Migración FNBC (`evidencias/salida_fnbc_migracion.txt`)

```
NOTICE:  Validación FD: F2 se cumple en los datos (0 responsables con >1 depósito)
NOTICE:  Verificación: original=3 vista=3 perdidas=0 espurias=0
COMMIT
NOTICE:  Salvaguarda F1 OK: F1 violada: el lote 501 ya tiene responsable en ese depósito
### Prueba de aborto: dato sucio (503,30,802) => 802 en depósitos 30 y 31, viola F2
ERROR:  F2 (responsable -> deposito) no se cumple en 1 responsables: migración abortada
### Objetos nuevos creados tras el aborto (esperado: ninguno)
(ninguno)
```

Las anomalías de la Parte 1 desaparecen: un responsable nuevo se registra sin lote, y cambiar de depósito a un responsable es un UPDATE de una sola fila.

3.2 Desnormalización: antes / después (`evidencias/explain_antes.txt`, `explain_despues.txt`)

	Antes: consulta original (4 tablas)	Después: `venta_categoria_dia`
Tiempo de ejecución	**26,99 ms** (otras corridas: 25,3 · 24,9 · 26,7 ms)	**0,128 ms** (otras corridas: 0,160 · 0,134 · 0,139 ms)
Nodo dominante	`Parallel Seq Scan on pedido` (descarta 66.478 filas por worker) + 570 `Index Scan` sobre `detalle_pedido`	`Bitmap Index Scan on pk_venta_categoria_dia` (fecha = hoy)
Buffers leídos	6.286	10
Resultado	Idéntico: Bebidas 1.236.848,83 · Postres 1.212.002,32 · Pizzas 5.200 · Empanadas 730 · Ensaladas 500	

La consulta es unas 200 veces más rápida. El costo pasa a las escrituras: por cada línea de pedido se hacen dos búsquedas por clave primaria y un upsert. La auditoría sobre la base migrada devolvió **0 filas desincronizadas**, también después de cada uno de los 7 casos de prueba. El control negativo detectó la corrupción (`total_fuente 5550.00` contra `total_redundante 5551.00`).

3.3 Permisos (`TP07 (Unidad 5)/docs/salida_verificacion_permisos.txt`)

```
-- ¿Quién puede leer usuario.contrasena?
   rolname
-------------
 admin_datos
-- Pruebas negativas
SET ROLE app_web;
SELECT contrasena ...  -> permission denied
INSERT ... rol='ADMIN' -> permission denied
DELETE FROM pedido ... -> permission denied
CREATE TABLE ...       -> permission denied
SET ROLE reportes_bi;
SELECT ... usuario     -> permission denied
```

```
-- EXECUTE sobre funciones sensibles
   rolname   | autenticar | resetear
-------------+------------+---------
 PUBLIC      | f          | f
 admin_datos | t          | t
 app_web     | t          | f
 reportes_bi | f          | f
 soporte_ops | f          | t
-- Default privileges: función nueva
PUBLIC puede ejecutar fn nueva: false
```

Las pruebas positivas también pasan. `app_web` registra un usuario (que queda con rol USUARIO) y crea un pedido, y el total se recalcula por trigger aunque la app no tenga permiso sobre `pedido.total`. `reportes_bi` lee `usuario_anon`.

3.4 Auditoría y simulacro de acceso indebido

El log de la prueba de auditoría (`docs/log_auditoria_prueba.txt`) registra quién se conectó, desde dónde, con qué método de autenticación, cada INSERT y UPDATE con sus valores, y el intento de conexión fallido (`FATAL: password authentication failed for user "app_web"`). No registra el SELECT ni el ROLLBACK, y esa es justamente la limitación que motiva pgaudit.

#	Credencial	Acción del simulacro	Corrida 1	Corrida 2 (corregida)
1–2	app_web	4 intentos fallidos de `fn_autenticar` y 1 exitoso sobre un cliente	4 fallos, éxito (id 10)	Bloqueado: los fallos de la corrida 1 seguían en la ventana de 15 min

3	soporte_ops	Lectura masiva de `usuario`	20.005 filas (columnas permitidas)	Ídem
4	soporte_ops	Leer `contrasena`	permission denied	permission denied
5–6	soporte_ops	`GRANT admin_datos` y `SET ROLE admin_datos`	ambos rechazados	ambos rechazados
7–8	soporte_ops → app_web	Resetear la clave de un usuario ADMIN y entrar con ella	funcionó: login como ADMIN	DENEGADO y registrado

El escalamiento a nivel del motor falló como corresponde. El simulacro, en cambio, encontró dos defectos que la revisión estática había pasado por alto:

1. `fn_resetear_password` permitía tomar una cuenta ADMIN de la aplicación. Se corrigió y se verificó en la corrida 2.
2. Cuando `fn_autenticar` se llama con literales, el log crudo registra **la contraseña en claro** en la línea `STATEMENT`. Con consultas parametrizadas no ocurre: `prueba_fuga_claves_log.txt` muestra 1 aparición con literales y 0 con `$1, $2`. Esto confirma que el propio log es un activo sensible.

Reconstrucción desde el log anonimizado y contención (`docs/informe_incidente.md`). El equipo revisó la reconstrucción contra la secuencia efectivamente ejecutada y la validó. El log vincula la lectura masiva, los intentos de escalamiento y el reset del ADMIN con la credencial `soporte_ops`. Medidas adoptadas por el equipo:

- deshabilitar `soporte_ops` y cortar sus sesiones;
- cambio de clave controlado del ADMIN afectado y del cliente cuya clave se adivinó;
- uso obligatorio de consultas parametrizadas;
- **no** bloquear la IP de origen, porque es la de la propia aplicación y bloquearla dejaría la tienda fuera de servicio sin frenar al actor.

3.5 Simulacro de desastre (restauración contra reloj)

Escenario	Datos	RTO observado	RPO	Validación
TP8 de la cursada (`restore_20261028_190502.log`)	Base mínima de TP1	1,5 s	1,2 min	Conteos iguales, 0 productos y 0 detalles inválidos
Segunda entrega (`restore_poblada_20261110_230140.log`)	≈670.000 filas (400.008 detalles)	**2,16 s** (createdb + `pg_restore -j 4` + validación)	17 min de antigüedad del respaldo; 0 filas perdidas	5 / 20.005 / 50.012 / 200.003 / 400.008 en origen y destino; 0 inválidos

Informe post-incidente. Con datos reales el RTO sigue siendo de segundos: el dump comprimido pesa 6 MB y la mayor parte del tiempo se va en reconstruir índices. El RPO depende de la frecuencia del respaldo: con el respaldo diario programado, el peor caso es perder 24 horas de pedidos. Si el negocio no lo tolera, el paso siguiente es archivar el WAL para poder recuperar a un punto en el tiempo. El análisis completo del primer simulacro está en `TP08 (Unidad 6)/Parte5_Informe/informe_u6.pdf`.

3.6 Sincronización móvil (`TP10 (Unidad 8)/movil/simulador.py`)

Caso	Situación	Resultado
C2	Subió el precio en el servidor	CONFLICTO; tras aceptar el precio nuevo → SIMULADO_APROBADO
C3	Se piden más unidades de las que hay en stock (30 contra 15)	RECHAZADO
C4	Producto dado de baja lógica en el servidor	RECHAZADO; la lápida queda en el caché y el producto sale del catálogo visible
C5	Borrador sin cambios	SIMULADO_APROBADO (no confirma la venta ni toca el stock)
C6	Reenvío de la misma solicitud	Rechazado por UUID duplicado; el estado no cambia (idempotente)

4. Decisiones de diseño

- **Refactorización.** Se descompuso sobre la dependencia que viola la FNBC y se aceptó perder la preservación de F1 a cambio de eliminar las anomalías. Esa pérdida se compensa con un trigger, en lugar de quedarse en 3FN. La migración

es conservadora: el esquema heredado no se borra hasta que la aplicación lea del nuevo.

- **Desnormalizaciones.** Cada caso tiene una justificación medida y una salvaguarda:

Estructura	Por qué	Mecanismo y salvaguarda
`venta_categoria_dia`	Panel en tiempo real, consultado varias veces por minuto: de 27 ms a 0,13 ms	Triggers por delta en la misma transacción que la escritura; TRUNCATE prohibido; auditoría; reversible con DROP, porque la fuente de verdad no cambia
`mv_facturacion_categoria_mes`	Reporte mensual que tolera desfasaje	REFRESH programado, con índice único para `CONCURRENTLY`
`pedido.total`, `detalle_pedido.subtotal`	Se leen en cada listado de pedidos	Triggers, más un CHECK de coherencia del subtotal agregado en el code review (CR-01)

  Para el top 5 diario se descartó la vista materializada: cada REFRESH volvería a recorrer las cuatro tablas y entre refrescos el panel mostraría datos viejos.

- **Roles y permisos.**
  - Denegar por defecto, incluso sobre las tablas futuras: una tabla nueva puede contener datos personales, así que se concede explícitamente después de actualizar la spec.
  - Las operaciones sensibles van encapsuladas en funciones `SECURITY DEFINER` con `search_path` fijo. La aplicación nunca lee el hash de la contraseña: solo pregunta si coincide.
  - Los triggers que escriben tablas derivadas también corren como dueño. Así no hubo que darle a la aplicación permiso para modificar totales.
  - `admin_datos` administra el esquema sin ser superusuario.
  - La credencial humana de soporte se audita completa, mientras que la cuenta técnica de la aplicación solo registra modificaciones (volumen y fuga de datos).
- **Documental vs. relacional.** PostgreSQL sigue siendo el sistema de registro de pedidos. Crear un pedido requiere una transacción que abarca varias tablas (`sp_crear_pedido` bloquea el producto con `FOR UPDATE`, descuenta stock y registra líneas y total) e integridad referencial con el catálogo y los usuarios. MongoDB sirve para la lectura del «pedido completo» en una sola operación, pero solo como copia de lectura alimentada desde PostgreSQL: el experimento de divergencia mostró que dos copias escritas por separado se desincronizan.
- **Móvil.** «El móvil propone, el servidor dispone.» SQLite funciona solo como caché y cola en el borde, sin decidir nunca precio, stock ni ventas. Los conflictos se resuelven con una aceptación explícita del usuario, nunca de forma automática.

5. Herramientas de IA utilizadas

La Declaración de Uso de IA (DUIA) se presenta por separado. Esta sección resume la bitácora y las propuestas de IA aceptadas y descartadas.

Unidad	Herramienta	Finalidad · prompt o spec	Propuestas aceptadas / descartadas y justificación
U4	Claude Code	Scripts de migración, reversión y desnormalización · spec `TP06 (Unidad 4)/specs/kiro/`	**Aceptada:** tabla resumen con triggers. **Descartada:** vista materializada, porque no cumple el requisito de tiempo real. **Agregadas tras la revisión:** validación de F2 sobre los datos, trigger de F1, prohibición de TRUNCATE y control negativo en la auditoría
U5	Claude Code	Roles, funciones de autenticación, auditoría, simulacro y anonimizador · spec `docs/kiro_roles_spec/`; registros en `BITACORA.md`	**Descartada (permiso de más):** `ALTER DEFAULT PRIVILEGES … GRANT SELECT ON TABLES TO rol_app_lectura`. **Corregida (sentencia incorrecta):** `REVOKE … IN SCHEMA` no tenía efecto, lo que se detectó porque `\ddp` salió vacío en la verificación. **Elegida** la alternativa de triggers `SECURITY DEFINER` sobre dar `UPDATE(total)` a la app. **Diferidos** como trabajo futuro: el control por fila (RLS) para soporte y para la app
U6	OpenCode	Scripts de respaldo, mantenimiento y restauración · `SPEC_U6.md`	Cada comando se revisó antes de ejecutarlo; detalle en el informe de U6

U7	OpenCode / IA	Modelado, consultas y laboratorio · `TP09 (Unidad 7)/ia/`	Las afirmaciones de la IA sobre atomicidad y transacciones se contrastaron con la documentación oficial y con pruebas ejecutadas
U8	OpenCode / Kiro	Esquema SQLite, semáforo y code review · `docs/kiro/`, `bitacora_ia.md`	**Descartada (CR-02):** restaurar con `--clean` de forma desatendida en producción. **Aceptadas:** CR-01 (CHECK de subtotal), CR-03 (`Decimal128`) y CR-04 (STRICT)
Cierre	Claude Code	Revisión de la entrega y code review de los scripts nuevos (CR-05 a CR-11)	Se detectó que CR-01 figuraba como corregido pero no estaba versionado: se agregó el script y se probó. Todas las decisiones de CR-05 a CR-11 fueron revisadas y aprobadas por el equipo

Caso más significativo de error de la IA. El `REVOKE EXECUTE … IN SCHEMA` parecía correcto y corría sin error, pero no hacía nada: en PostgreSQL, los privilegios por defecto definidos a nivel de esquema solo pueden *agregar* permisos. Se detectó porque la verificación no se quedó en que el script corriera: se creó una función nueva y se consultó su ACL.

Protocolo de datos. Solo se trabajó con datos sintéticos. Los logs se anonimizaron antes de analizarlos, y las contraseñas de las cuentas de prueba se generaron al azar y no están versionadas. Todo script generado con IA se leyó y se probó dentro de una transacción reversible o sobre una copia, con respaldo previo.